WhatsApp WooCommerce AI Agent Implementation Specification

Agent ready engineering handoff | Version 1.0 | 28 October 2026

Prepared for Amigoo IT Solutions | Proposed stack Node.js Express, MySQL, Meta WhatsApp Cloud API, WhatsApp Flows, OpenAI and existing WooCommerce

1 Purpose and implementation directive

Build a production grade WhatsApp sales assistant for one existing WooCommerce store. Customers discover products through natural language chat, complete a structured WhatsApp Flow, pay using the existing WooCommerce gateway, and receive verified order updates. The store remains the authoritative source of product, pricing, stock, tax, shipping, payment and fulfillment information.

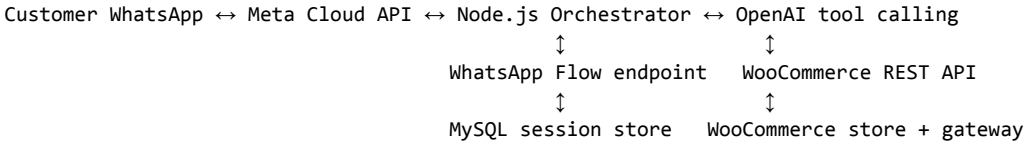
The coding agent must implement the MVP in stages, run automated tests, document setup, and never change production WordPress or payment settings without explicit approval. Use staging and sandbox accounts until acceptance criteria are met.

2 Scope and decisions

In scope for MVP	Out of scope unless separately approved
Inbound WhatsApp chat and AI product search	Cold marketing campaigns and bulk messaging
Dynamic WhatsApp Flow for checkout details	Native in chat payments without confirmed eligibility
Live WooCommerce stock and variant lookup	Replacement of WooCommerce checkout or database
Pending order and secure payment link	Refund initiation by AI
Verified order status notifications	Autonomous price changes and custom discounts
English and Telugu text, human handoff	Voice transcription and media understanding

Assumptions to confirm before coding: store URL, WooCommerce version, payment gateway, shipping/tax plugins, type of products, WhatsApp WABA and business phone eligibility, hosting domain, expected volume, and data retention requirements.

3 Architecture and ownership



- WooCommerce is source of truth for catalog, inventory, final price, tax, shipping, order state and paid state.
- Node.js owns routing, authenticated webhook processing, conversational state, Flow sessions, idempotency and integration audit logs.
- The AI may recommend products and request read only tools. It may not fabricate prices, order identifiers or success states.
- A minimal private WordPress plugin may be needed for native checkout calculations and a canonical order payment URL.

4 Required external configuration

1. Create staging WooCommerce instance and verify existing checkout, tax, coupons, variable products, stock and shipping.
2. WooCommerce > Settings > Advanced > REST API: issue dedicated scoped read write credentials for integration user. Keep secrets only on backend.
3. Create Meta developer app with WhatsApp Business Platform; connect WABA and test number; provision appropriate system user token and permissions.
4. Configure HTTPS GET challenge and signed POST webhook at /api/webhooks/whatsapp, subscribe to messages; test with a real inbound message.
5. Create WhatsApp Flow through Meta Flow Builder/Flow API, upload valid Flow JSON, configure data endpoint, register encryption public key, test, publish, and record Flow ID.
6. Configure WooCommerce order.updated (and relevant status) webhooks with HMAC secret and verify incoming raw body signatures.
7. Use sandbox gateway credentials and approved WhatsApp templates where business initiated messages fall outside the customer service window.

5 Implementation milestones

Stage	Deliverable	Exit gate
M1	Environment, webhook verification, WooCommerce read API	Hi message reaches staging service; product query returns real data
M2	AI tool calling, catalog retrieval, conversation storage	No invented product/price; language and handoff tested
M3	Flow JSON, encrypted data exchange, session binding	Flow works with selected product and delivery validation
M4	Checkout calculation, pending order, payment URL	One valid Woo order from one submission
M5	Payment/status sync, retries, notifications	Paid state only after verified gateway/Woo state
M6	Observability, security review, UAT, deployment	All acceptance cases pass; rollback documented

6 Repository structure

```
src/config/{env,db,woo,meta}.js
src/routes/{whatsapp,flows,woocommerce,health}.js
src/services/{agent,catalog,cart,checkout,orders,notifications}.js
src/adapters/{metaCloud,wooRest}.js
src/security/{metaSignature,wooSignature,flowCrypto}.js
src/jobs/{retryNotifications,reconcileOrders}.js
src/db/migrations/  tests/{unit,integration,e2e}/  docs/{setup,runbook}.md
```

7 Backend API contracts

Route	Contract	Authentication
GET /api/webhooks/whatsapp	Meta hub challenge response	Verify token
POST /api/webhooks/whatsapp	Incoming messages, Flow completion, statuses	Meta signature on raw payload

POST /api/flows/data	Encrypted Meta data exchange; screen routing and payload	Flow cryptography and session checks
POST /api/webhooks/woocommerce	Order event; enqueue reconciliation	WooCommerce HMAC signature
GET /api/health	Non-sensitive liveness/readiness	Internal monitoring
Internal checkout service	Validate cart, shipping and amounts; create order	Server only; never public unauthenticated

Flow data exchange is a separate protocol from the ordinary WhatsApp message webhook: decrypt the inbound AES key and payload according to Meta documentation, route based on action/screen/data, encrypt the response, and handle protocol errors without leaking private keys. Do not expose Flow endpoint inputs directly to WooCommerce.

8 AI tool interface

Tool	Inputs	Hard constraints
search_products	query, category, limit	Published products only; bounded result set
get_product	product_id	Return authoritative price and image URLs
get_variations	product_id	Only purchasable variations
check_availability	product_id, variation_id, qty	Recheck near order placement
start_checkout	customer/session, selected lines	Creates Flow session, not order
get_order_status	verified customer session, order id	Enforce ownership; do not reveal others' orders
request_human	conversation id, reason	Stop bot replies until handoff released

AI system policy: use tool results to answer product questions; label unavailable details as unknown; ask clarification for ambiguous variants; avoid assurances about delivery before shipping calculation; never execute payment or refund solely from a conversational request. Make order creation a deterministic backend operation only after explicit customer confirmation.

9 WhatsApp Flow screen contract

Screen	Data	Validation
PRODUCT	product_id, variation_id, quantity	Session approved SKU and active variation; quantity within stock
CUSTOMER	name, phone, optional email	Normalize WhatsApp ID; validate required billing data
DELIVERY	address lines, city, state, country, postcode	PIN format, serviceability, available shipping methods
REVIEW	line items, shipping, tax, discounts, total	All values recalculated server side; final explicit confirmation

Use opaque random flow_token and an expiring flow_session record. The token maps to customer, conversation, cart revision and selected product IDs. On every screen transition validate token expiry and ownership. On submit lock the session, recalculate totals, and atomically record the one order ID; repeated submissions return the same order/payment reference.

10 Checkout and payment state machine

```
CART_OPEN → FLOW_ACTIVE → QUOTE_VALIDATED → ORDER_PENDING → PAYMENT_PENDING
                                                  ↘ EXPIRED/CANCELLED
```

```
PAYMENT_PENDING → PAID_VERIFIED → PROCESSING/COMPLETED → NOTIFIED
PAYMENT_PENDING → FAILED/RETRYABLE
```

8. Recalculate final totals inside WooCommerce-compatible checkout logic. Do not compute authoritative totals in the LLM.
9. Create one WooCommerce pending order with billing, shipping, product and variant lines, selected shipping, coupon/tax results and source metadata.
10. Persist checkout_session_id ↔ Woo order ID with a uniqueness constraint; retries must reuse the existing order.
11. Generate canonical WooCommerce order-payment URL (e.g. using WC_Order::get_checkout_payment_url in a secured plugin endpoint).
12. Send URL in chat. The existing gateway processes payment and records paid state in WooCommerce.
13. On signed WooCommerce webhook, retrieve current order from WooCommerce and verify paid state/transaction before notifying.
14. Keep failed or abandoned payments unpaid. Permit retry on same payable order where business rules allow; reconcile missed events.

11 Suggested MySQL schema

Table	Important fields / constraints
wa_customers	id, wa_id UNIQUE, wc_customer_id, locale, consent_at
wa_conversations	id, customer_id, state, handoff_status, last_message_at
wa_messages	id, meta_message_id UNIQUE, conversation_id, direction, payload_redacted, status
wa_carts / wa_cart_items	cart id, revision, item product_id, variation_id, quantity
wa_flow_sessions	id, flow_token UNIQUE, customer_id, cart_revision, expires_at, status
wa_order_links	checkout_session_id UNIQUE, flow_session_id UNIQUE, wc_order_id UNIQUE, payment_state
wa_events	provider, external_event_id, received_at, processed_at, status; UNIQUE(provider,external_event_id)
wa_handoffs	conversation_id, agent_id, reason, started_at, closed_at

Use transactions, unique indices and state transitions to prevent duplicate orders. Retain only needed PII; redact phone, tokens, addresses and payment metadata from application logs. Define data retention and deletion procedures before launch.

12 Operational and security requirements

- Verify signatures on raw webhook request bytes; reject invalid requests. Respond quickly, enqueue work, retry idempotently.
- Use HTTPS, least privilege API keys, rotating secrets and private-key storage; never send API secrets or raw billing data to LLM unnecessarily.
- Rate limit inbound processing and AI tool invocations; apply per-customer spending and volume guardrails.
- Implement structured logs with correlation IDs, metrics for webhook failures, Flow completion, order mismatch and payment reconciliation.
- Respect Meta template/24-hour messaging restrictions and relevant privacy and consent requirements.
- Reconcile periodically: compare pending and recently paid Woo orders with local links, without overwriting verified Woo state.
- Deploy with PM2 behind Nginx, environment secret management, separate staging/prod tokens, backups, and rollback runbook.

13 Acceptance tests for the coding agent

ID	Scenario	Expected outcome
A01	Inbound Hi and product query	Authenticated delivery, correct live products, no invented prices
A02	Variable product selection	Exact variation ID preserved through Flow and order
A03	Stock changes after recommendation	Order blocked or requoted before confirmation
A04	Invalid PIN or unavailable shipping	Flow displays clear correction, no order
A05	Coupon/tax/shipping calculation	Displayed total equals WooCommerce payable total
A06	Flow submitted twice / webhook retried	Exactly one Woo order and one payment reference
A07	Payment unsuccessful / customer says paid	Order remains unpaid; no false confirmation
A08	Verified payment success	Woo order paid, single WhatsApp confirmation
A09	Webhook out of order / missed	Safe reconciliation, no regressed paid state
A10	Customer queries another customer order	Access denied and no PII leakage
A11	Telugu and English conversation	Correct responses preserving factual product data
A12	Human handoff	Bot stops responding and thread assigned to operator

14 Definition of done and delivery package

- Runnable Node.js repository with source, migrations, example environment file (placeholders only), Docker or PM2 deployment documentation.
- Meta Flow JSON with documented screen schema, encrypted endpoint implementation, publish procedure and test evidence.
- WooCommerce connector and optional minimal WordPress plugin for checkout quote and secure payment URL.
- Automated unit, integration and end-to-end tests plus a UAT results sheet covering A01–A12.
- Operational runbook covering setup, key rotation, webhook retries, reconciliation, support handoff and rollback.
- Production deployment only after staged payment tests, user approval and observed monitoring.

15 Coding agent instruction

Implement milestones M1 through M6 sequentially. Before each milestone, inspect the repository and clarify missing configuration values; do not invent credentials, payment methods or URLs. Work in staging using mocked provider adapters wherever credentials are unavailable. Create tests before enabling external writes. Show changed files, commands run, test results and outstanding risks at each milestone. Do not claim production readiness until A01–A12 pass. Never log secrets or call live charging APIs without explicit authorization.

16 Primary references

- Meta WhatsApp Flows: https://developers.facebook.com/documentation/business-messaging/whatsapp/flows

- Meta Flow endpoint: https://developers.facebook.com/documentation/business-messaging/whatsapp/flows/guides/implementingyourflowendpoint
- Meta send a Flow: https://developers.facebook.com/documentation/business-messaging/whatsapp/flows/guides/sendingaflow
- WooCommerce REST API: https://developer.woocommerce.com/docs/apis/rest-api/
- WooCommerce webhooks: https://developer.woocommerce.com/docs/apis/rest-api/v3/webhooks/
- WooCommerce order code reference: https://woocommerce.github.io/code-reference/files/woocommerce-includes-class-wc-order.html

Implementation note: API versions, Meta capabilities, payment eligibility and pricing can change. Revalidate provider documentation at implementation time. This specification intentionally prefers the existing WooCommerce payment gateway for MVP; native WhatsApp Payments is a separately gated enhancement.